

HOPE LEARNING

Item Based Recommendation System: Technical Writeup

(by Dr. N. Dilip Raja)

This document provides a comprehensive explanation of the Item-Based Recommendation System code, which implements collaborative filtering to predict movie ratings based on item (movie) similarity rather than user similarity. The system recommends movies by analyzing how similar one movie is to other movies that a user has already rated highly.

File Information

- **Source File:** MultipleFiles/2 Item Based Recommendation System.py
- **Purpose:** Item-Based Collaborative Filtering for Movie Recommendations
- **Dataset:** MovieLens 100k
- **Application:** Recommending movies based on item similarity

Step-by-Step Processes

1. Import Required Libraries

The code begins by importing essential Python libraries for data manipulation and computation:

- **pandas:** Used for data frame operations and handling structured data
- **numpy:** Used for numerical computations and array manipulations
- **sklearn.metrics.pairwise:** Provides the pairwise_distances function for calculating cosine similarity

2. Load Dataset

The system loads the MovieLens 100k dataset containing user movie ratings:

- **File:** ml-100k/u.data
- **Separator:** Tab-delimited file (`\t`)
- **Column Names:** User_id, Movie_id, Ratings, Unique_timestamp
- **Encoding:** Latin-1 encoding
- This dataset contains multiple ratings from various users for different movies

The ratings data is loaded into a pandas DataFrame for processing.

3. Identify Number of Items

The code determines the total number of unique items (movies) in the dataset:

- Uses `Ratings.Movie_id.unique().shape[0]` to count distinct movies
- Displays: *"There are {N_Items} in the dataset"*

- This information is crucial for setting up the item similarity matrix

4. Create Pivot Table

A user-movie rating matrix (pivot table) is created:

- **Index:** User_id
- **Columns:** Movie_id
- **Values:** Ratings
- This creates a sparse matrix where rows represent users and columns represent movies they have rated

The pivot table allows the system to analyze ratings from the perspective of items rather than users.

5. Data Preprocessing

The pivot table undergoes preprocessing:

- **NaN Replacement:** Missing values (NaN) are replaced with 0 using `Pivot_table.replace(np.nan, 0)`
- This conversion ensures the similarity calculations work properly without missing value errors

The preprocessed data is stored in **Data_matrix** for subsequent operations.

6. Calculate Item Similarity

The code computes cosine similarity between all items (movies):

- Uses `sklearn.metrics.pairwise.pairwise_distances()` with `metric='cosine'`
- **Key Difference:** The matrix is transposed (**Data_matrix.T**) to calculate item-item similarity instead of user-user similarity
- This creates a similarity matrix where each cell represents how similar two movies are based on user ratings
- Higher cosine similarity indicates movies that tend to be rated similarly by the same users

The resulting **Item_similarity** matrix shows relationships between all movie pairs.

7. Rating Prediction Function

A prediction function is defined to forecast ratings:

```
def prediction(Ratings, Similarity, type = 'item'):
    Item_ratings_mean = Ratings.mean(axis = 1).to_numpy()
    Ratings_diff = Ratings - Item_ratings_mean[:, np.newaxis]
    Predict = Ratings.dot(Similarity) / np.array([np.abs(Similarity).sum(axis=1)])
```

```
return Predict
```

This function:

- Calculates the mean rating for each item
- Computes rating differences from the mean
- Uses weighted similarity to predict ratings
- Returns a prediction matrix for all user-movie pairs

8. Generate Predictions

The prediction function is executed:

- Creates an **Item_prediction** matrix containing predicted ratings for all users and movies
- Each value represents the predicted rating a user might give to a particular movie
- This matrix is used to identify highly-rated movie recommendations

9. Select Input Item

The code designates a specific movie for recommendation analysis:

- **Input_items = 34:** Movie ID 34 is selected as the reference item
- The system will find movies similar to this item and recommend them to users who have watched Movie ID 34

10. Convert Similarity to DataFrame

The item similarity matrix is converted to a pandas DataFrame:

- **Item_Sim_Table = pd.DataFrame(Item_similarity)**
- This conversion allows easier manipulation and sorting of similarity values
- Each row and column represents a movie, with similarity values in the cells

11. Find Similar Items

The code identifies the most similar items to the input movie:

```
Sim_Input_Item = Item_Sim_Table[Input_items].sort_values(ascending = True).head(5).index
```

- Sorts the similarity values for Movie ID 34 in ascending order
- Selects the top 5 most similar movies (lowest cosine distance = highest similarity)
- Returns the indices (Movie IDs) of these similar movies

12. Extract User Lists for Similar Items

The code retrieves all users who watched each similar movie:

```
Sim_Movie_UserID_List = []
```

```
for Sim_Item in Sim_Input_Item:
```

```
Sim = list(Ratings[Ratings['Movie_id']==Sim_Item]['User_id'])
```

```
Sim_Movie_UserID_List.append(Sim)
```

- Iterates through each similar movie
- Finds all User IDs who rated that movie
- Stores users in a nested list structure

13. Flatten User List

The nested user list is flattened into a single list:

- Uses **itertools.chain.from_iterable()** to combine all nested lists
- Creates **Sim_Movie_UserID_Single_List** containing all users who watched any similar movie

14. Extract Unique Users

The code identifies unique users from the list:

- Uses **set()** to remove duplicate user IDs
- Creates **Unique_UserID_Similar_Item** containing distinct users who watched similar movies

15. Get Input Item's Users

Retrieves all users who watched the input movie (Movie ID 34):

```
Input_Item_Watched_UserID = list(Ratings[Ratings['Movie_id']==Input_items]['User_id'].values)
```

- Stores User IDs in **Input_Item_Watched_UserID** list

16. Filter Recommended Users

The code filters out users who have already watched the input movie:

```
Recom = []
```

```
for per_id in Unique_UserID_Similar_Item:
```

```
    if (per_id in Input_Item_Watched_UserID):
```

```
        pass
```

```
    else:
```

```
        Recom.append(per_id)
```

- Creates a list of recommended users who haven't seen the input movie yet
- These are potential users who could benefit from recommendations

17. Analyze Prediction Values

The code uses the prediction matrix to identify highly-rated movies:

- Converts **Item_prediction** to a DataFrame for easier access

- Iterates through recommended users
- Checks predicted ratings for the input movie
- Filters users with predicted ratings ≥ 1

18. Final Filtering

The code performs cross-checking to ensure accuracy:

- Verifies that recommended users aren't in the input movie's user list
- Confirms the final list of recommended movies/users

Summary

This Item-Based Recommendation System successfully:

1. Loads and preprocesses the MovieLens 100k rating data
2. Creates a user-movie rating matrix
3. Computes item-item similarity using cosine distance on the transposed matrix
4. Predicts ratings using weighted similarity scores
5. Identifies the most similar items to a given input movie
6. Finds users who watched similar movies but haven't watched the input movie
7. Predicts which users would rate the input movie highly (rating ≥ 1)